

Contents

Product Document	1
What we built	1
Who it is for	1
Why this matters	2
Core user journey	2
Key product decisions and tradeoffs	2
Shopify Catalog MCP, not Storefront API	2
Clarify only when missing-info changes the answer	2
One-line concierge prose, never headings or pipe tables	2
Per-chat seeded variation	3
Generative UI as first-class output	3
Free-tier model chain with fallbacks	3
Recommendation feed at / is non-AI	3
Guest auth as the dominant CTA	3
Scope	3
How success looks	4
Why this is hackathon-strong	4

Product Document

What we built

An AI shopping concierge that turns plain-language shopping intent into grounded product recommendations from the Shopify Global Catalog. The buyer types what they want, the agent asks clarifying questions only when the missing constraint actually changes the result set, then searches live Shopify catalog data, compares tradeoffs, and recommends products with reasoning the buyer can verify.

Live in this submission:

- Chat surface at /chat for the agent loop.
- Recommendation feed at / for catalog discovery without an AI call.
- Generative UI parts (product grid, comparison table, seller comparison, freshness verdict card, option chips, web-search evidence panel) rendered from tool outputs.
- Per-chat personality + seeded shuffle so two judges hitting the same demo see different orderings of the same catalog, without breaking factual grounding.

Who it is for

Two audiences, same surface:

1. Buyer. Knows what they want in plain English (“a gaming laptop under \$1500 with good battery life”) but does not want to wade through 50 results and 12 filters to compare. They want a short, defended recommendation and a click to buy.
2. Shopify merchant network. Cross-merchant catalog search means a buyer can land on the best variant across all merchants, not just one store. The merchant benefits from being discoverable inside an agent surface rather than only via classic search.

Why this matters

Classic search is keyword-shaped. Buyers are intent-shaped. The gap costs merchants conversions and costs buyers confidence. Three concrete failures we wanted to remove:

- Stale recommendations in fast-moving categories. A “gaming phone” search surfacing a Snapdragon 845 device is technically a match but a real-world wrong answer. Our freshness audit detects category + chipset + signals (Wi-Fi 7, BT 5.4, USB-C PD, etc.) and labels current-gen vs. previous-gen vs. legacy before the buyer commits.
- Comparison without context. Side-by-side spec tables that hide the actual decision. Our comparison output highlights the cheapest variant, surfaces the best seller, and the model is forced to name the tradeoff in its prose.
- Hand-waved compatibility claims. The model is constrained to refuse compatibility answers without tool evidence. If web/product lookup fails, it says it could not verify rather than guess.

Core user journey

1. Buyer describes intent in natural language.
2. Agent asks one follow-up only when a missing constraint materially changes the product set. Clarifications render as quick-reply chips.
3. Agent searches the Shopify Catalog MCP.
4. For fast-moving categories, agent runs `assessProductFreshness` and may counter-search with a current-gen hint in the same budget.
5. Agent presents 2–6 products as cards, compares 2–4 side-by-side on demand, or breaks down sellers per product.
6. Buyer clicks Buy on the cheapest in-stock variant. URL is the merchant’s own Shop Pay / variant / product page — no checkout proxy.

Key product decisions and tradeoffs

Shopify Catalog MCP, not Storefront API

Chose Shopify’s UCP-compliant Catalog MCP server over the per-store Storefront API. Tradeoff: we lose per-store branding control but gain cross-merchant search out of the box, plus a single `checkoutUrl` per variant that maps to Shop Pay. This is the closest fit to Track 1’s “across Shopify merchants” framing.

Clarify only when missing-info changes the answer

Earlier prompts forced a clarify menu on every turn. Judges reported it felt like a form. We softened to “clarify only when the missing choice materially changes the product set.” Tradeoff: occasionally the model picks one product class when two were plausible. Net better — the surface feels like a concierge instead of an interrogation.

One-line concierge prose, never headings or pipe tables

Free-tier LLMs love to emit markdown headings, bullet thickets, and pipe tables. They look like academic notes, not retail. Prompt + frontend stripper enforces: short prose only, native UI table for

comparisons, native cards for products. Tradeoff: extra defensive code in `assistant-response-
json.ts` to strip pipe tables and unwrap `{"responseText": "..."} JSON envelopes if the model leaks them.`

Per-chat seeded variation

Same query in two different chats should not produce identical orderings or identical opening sentences. A 32-bit `chatId` hash seeds: - a personality hint in the system prompt, - the shuffle of the long tail of search results (top-2 stable), - the homepage suggested-action chips, - the greeting copy. Tradeoff: more state to test, but reproducibility within a chat is preserved (same chat → same outputs across reloads).

Generative UI as first-class output

Tool outputs render as native components, not as model-emitted markdown. The model can lie in prose; it cannot fabricate a product card without a tool result. Tradeoff: more frontend surface area, but the UI grounding is the strongest defense against hallucination.

Free-tier model chain with fallbacks

Primary `z-ai/glm-4.5-air:free`, fallbacks `openai/gpt-oss-120b:free` and `meta-llama/llama-3.3-70b-instruct:free`. Direct DeepSeek `deepseek-v4-flash` is also wired. Tradeoff: rate limits during demos; mitigated by fallback chain and graceful degradation when the chosen model 429s.

Recommendation feed at / is non-AI

The homepage browses the live Shopify catalog with daily-rotated categories and 5-min/1h Redis caches. Zero LLM cost. Tradeoff: feed quality is bounded by raw catalog ranking, but it gives judges a “look at the product surface without spending a token” entry path before they open the chat.

Guest auth as the dominant CTA

Login and register both lead with “Continue as guest” so a judge can demo without signup friction. Tradeoff: anonymous chats are pruned aggressively and not personalized.

Scope

In scope:

- Cross-merchant product discovery via Shopify Catalog MCP.
- Conversational refinement (search → refine → showMore → compare).
- Freshness audit for 19 fast-moving categories.
- Web search evidence for reviews and compatibility questions.
- Generative UI for products, comparisons, sellers, freshness verdicts, web evidence.
- Recommendation feed surface for non-chat discovery.
- Guest mode.

Out of scope (for this submission):

- Multi-currency checkout. We display variant prices in their native currency from Shopify and convert verbally in prose, but the buyer follows the merchant's own checkout flow.
- Saved carts across chats.
- Personalization based on prior chats (no profile / no recommendations across sessions).
- Tracks 2–5 (registry slots exist as stubs).
- Native mobile apps (responsive web only).
- Image search / “find similar by photo”.

How success looks

- Time-to-first-good-recommendation under 30 seconds against the live Shopify Catalog MCP.
- Buyer can compare 2–4 products without scrolling through a markdown wall.
- Stale gear in fast-moving categories is flagged before the buyer commits.
- Every product link goes to a real merchant page; no dead Buy buttons.
- Two judges hitting the same demo see different conversational tone without different facts.

Why this is hackathon-strong

- Track 1 framing literally calls out cross-merchant discovery → Catalog MCP is the right primitive.
- The freshness layer is the differentiator. Generic agent demos do not address generation drift in electronics. Ours does, with a curated chipset table and a counter-search hint.
- The generative UI grounding is a real anti-hallucination mechanism, not a UX flourish — the model cannot show a card it did not earn from a tool.
- The submission includes a non-chat surface (the feed) and a chat surface, which mirrors how a real Shopify merchant would deploy this: a browse layer plus an agent layer over the same catalog.